Symbolic Operators

—

User Guide

Joshua Althüser

2026

1 Prerequistes

Valid installations of

- C++ 20 and a functioning compiler (tested with g++ 13.3.0 on WSL and g++ 11.5.0 on Red Hat)
- (recommended) CMake 3.30 or newer (tested with version 3.31.8)
  — https://cmake.org/
  — used for easy installation running of tests
- (optional) boost (tested with version 1.78.0)
  — https://www.boost.org/
  — used for serialization, saving, and reading the results. If one does not want to delegate these tasks to boost, one can of course implement it.

2 Overview

The `symbolic_operators` library serves as a lightweight utility for commuting the canonical creation and annihilation operators. The library supports both fermionic and bosonic types of operators. Additionally, if a higher-order expression, such as $c_1^\dagger c_2^\dagger c_3 c_4$, is given, the program can reduce its expectation value to products of bilinear expressions using Wick's theorem.

To keep this document short, we do not list or explain all functions here; instead, we only discuss the most important attributes and functions. Please view the source files themselves. Everything user-facing is defined in the namespace `mrock::symbolic_operators`. The namespace `mrock::symbolic_operators::detail` contains various functions that are used under the hood. Most functions of the classes are also intended to be used internally, though, if desired, they can be used by the user as well.

3 Overview of the classes

3.1 AbstractTerm

`AbstractTerm` is a class template with the template parameter `tOperatorType`. The class template serves as a parent of the `Term` and `WickTerm` classes; both inherit publicly from it. `AbstractTerm` is not intended to be used directly. Instead, it contains the attributes and functionality used by both classes. The attributes are

- The multiplicity of the term, represented by `IntFractional multiplicity`

- The list of coefficients, stored in `std::vector<Coefficient> coefficients`
- The summation structure, stored in `SumContainer sums`
- The Kronecker deltas for momenta, stored in `std::vector<KroneckerDelta<Momentum>> delta_momenta`
- The Kronecker deltas for indices, stored in `std::vector<KroneckerDelta<Index>> delta_indizes`
- The operators contained in the term are stored in `std::vector<tOperatorType> operators`; if this vector is empty, the term is treated as the identity operator

3.2 Coefficient

`Coefficient` represents one symbolic coefficient that can appear in a term. It stores the coefficient name, the associated momenta and indices, and a set of symmetry flags indicating whether the coefficient is inversion-symmetric, real, or subject to additional custom symmetries. This class is used for hopping amplitudes, interactions, and similar objects that enter the symbolic expressions. The attributes are

- The coefficient name, stored in `name`
- The list of momenta, stored in `MomentumList momenta`
- The index wrapper, stored in `IndexWrapper indizes`
- An optional custom symmetry callback, stored in `custom_symmetry`
- Symmetry flags such as `inversion_symmetry`, `is_symmetrized_interaction`, `Q_changes_sign`, `is_real`, and `is_daggered`

3.3 Fractional

`Fractional` is a small template for representing rational numbers with an integer numerator and denominator. It supports arithmetic and comparisons, conversion to built-in numeric types, and reduction to lowest terms. In this library, it is mainly used for multiplicities and other bookkeeping quantities. The attributes are

- The numerator, stored in `numerator`
- The denominator, stored in `denominator`

`IntFractional` is a template specialization for `int`, i.e.,

```
typedef Fractional<int> IntFractional;
```

Integers can be implicitly converted to objects of this class. That is, all of the following examples are valid

```
Term t1(1); // implicitly convert int to IntFractional
Term t2(IntFractional(1)); // use the typedef
Term t3(Fractional<int>(1)); // directly use the class template
Term t4(IntFractional(1,2)); // represents 1/2
```

3.4 Index

`Index` is an enum class that represents the symbolic labels used for spin, orbital, or other internal indices. It includes predefined values such as spin-up and spin-down states, as well as general-purpose labels for use in symbolic expressions.

3.5 IndexWrapper

`IndexWrapper` wraps a collection of `Index` values and is used to describe the internal indices of an operator or a coefficient. In addition to storing the indices, it provides convenience constructors and comparison operators, making it easy to pass around spin or orbital labels. The attributes are

- The wrapped list of indices, stored in `indizes`

3.6 KroneckerDelta

`KroneckerDelta` is a lightweight template that represents a Kronecker delta between two objects of the same type. It is used throughout the library for enforcing momentum and index constraints, and the helper function `make_delta` provides a convenient way to construct such objects. The attributes are

- The first entry of the delta, stored in `first`
- The second entry of the delta, stored in `second`

3.7 Momentum

`Momentum` represents a symbolic momentum expression as a collection of `MomentumSymbol` objects. It supports addition and subtraction-like algebra, the optional inclusion of the special momentum Q, and helper functions for sorting, flipping, and removing contributions. Strings such as `"3k+l-p"` can also be parsed into a momentum object. The attributes are

- The list of momentum symbols, stored in `momentum_list`
- The flag indicating whether the special momentum Q is present, stored in `add_Q`

3.8 MomentumList

`MomentumList` is a thin wrapper around a vector of `Momentum` objects. It is mainly used for coefficients that depend on several momenta, such as interaction vertices, and provides convenience methods for applying global operations to all entries. The attributes are

- The stored list of momenta, stored in `momenta`

3.9 MomentumSymbol

`MomentumSymbol` stores one symbolic momentum contribution together with its integer prefactor and name. This is the most basic building block of the library's momentum algebra. The attributes are

- The integer prefactor, stored in `factor`
- The momentum name, stored in `name`

3.10 Operator

`Operator` represents a single fermionic or bosonic creation or annihilation operator. It stores the momentum, the relevant indices, whether it is daggered, and whether it is a fermion or a boson. The class, therefore, forms the basic unit that is later combined into larger terms. The attributes are

- The momentum of the operator, stored in `momentum`
- The associated indices, stored in `indizes`
- The dagger flag (whether or not the operator is a Hermitian conjugate), stored in `is_daggered`
- The fermion flag, stored in `is_fermion`

3.11 OperatorType

`OperatorType` is an enum class that labels the different kinds of Wick operators that can appear after applying Wick's theorem. It is used to distinguish number-like operators, pair operators, and other types of symbolic contractions.

3.12 SumContainer

`SumContainer` collects the symbolic sums appearing in a term. It stores sums over momenta and over spins separately and provides methods to append additional sums or to check whether a momentum or spin sum is present. The attributes are

- The momentum sums, stored in `momenta`
- The spin sums, stored in `spins`

3.13 SymbolicSum

`SymbolicSum` is a generic helper class for storing one or more symbolic summation indices. It is templated on the index type and can be used for both momentum sums and spin sums. Its main purpose is to track which indices are being summed over and to make such information available in a compact form. The attributes are

- The list of summed indices, stored in `summations`

3.14 Term

This class inherits publicly from `AbstractTerm` and specifies the template as `Operator`. `Term` is used to represent a string of creation and annihilation operators. Fermionic and bosonic operators may be mixed. It does not define any new attributes beyond the inherited members of `AbstractTerm`; the user-facing data is therefore the same multiplicity, coefficient, sum, delta, and operator storage inherited from the base class. However, it defines a set of new functions and constructors.

This class is the heart of the library's outward-facing part. Almost anything a user might want to do will involve creating `Term` objects. Therefore, there are many ways to initialize such an object. See the source file for a list of them.

3.15 TermLoader

`TermLoader` is a small utility class for loading precomputed `WickTerm` objects from disk. It stores vectors of `WickTermCollector` objects for the matrices M and N appearing in the iterated equations of motion method[1]. This class should make it convenient to load and manage larger sets of symbolic terms. The attributes are

- The loaded `WickTerm` objects for the matrix M, stored in `M`
- The loaded `WickTerm` objects for the matrix N, stored in `N`

3.16 WickOperator

`WickOperator` represents an operator that appears in a Wick contraction. It stores the operator type, Hermitian conjugate ("dagger") status, momentum, and indices, and can be converted back to ordinary `Operator` objects when needed. This class is used internally when applying Wick's theorem to higher-order expressions. The attributes are

[1] J. Althüser and G. S. Uhrig, Phys. Rev. B **109**, 205153 (2024), https://doi.org/10.1103/PhysRevB.109.205153

- The operator type, stored in `type`
- The dagger flag (whether or not the operator is a Hermitian conjugate), stored in `is_daggered`
- The momentum of the operator, stored in `momentum`
- The associated indices, stored in `indizes`

3.17 WickOperatorTemplate

`WickOperatorTemplate` provides the logic for deciding whether a given pair of ordinary operators can be interpreted as a specific Wick contraction. It stores index comparison rules and the expected momentum difference, and it returns a template result describing the allowed operator structure.

3.18 WickSymmetry

`WickSymmetry` is an abstract base class for symmetry operations that can be applied to `WickTerm` objects after contraction. The library already provides concrete implementations such as `SpinSymmetry`, `InversionSymmetry`, and `PhaseSymmetry`, which simplify expressions by exploiting symmetry relations.

3.18.1 SpinSymmetry

`SpinSymmetry` changes all spins in a `WickTerm` object to `Index::SpinUp` (↑), effectively enforcing spin symmetry and simplifying expressions that do not depend on the spin orientation.

3.18.2 InversionSymmetry

`InversionSymmetry` flips momenta so that the first momentum in a term is always treated in a canonical orientation, which helps identify equivalent terms under inversion.

3.18.3 PhaseSymmetry

`PhaseSymmetry` removes the dagger from operators of selected `OperatorType` values, making the corresponding expectation values real or phase-symmetric.

3.19 WickTerm

This class inherits publicly from `AbstractTerm` and specifies the template as `WickOperator`. `Term` is used to represent a string of expectation values, represented via `WickOperator` objects. The main additional attribute is `temporary_operators`, which is used internally to transform a string of `Operator` objects into `WickOperator` objects. Objects of this class are usually not created by the user directly, but obtained as a result of the `wicks_theorem` function. The attributes are

- The inherited term data from `AbstractTerm`
- The temporary operator list, stored in `temporary_operators`

3.20 WickTermCollector

`WickTermCollector` is a simple container for collecting many `WickTerm` objects. It supports the usual arithmetic-style accumulation operations and is therefore convenient when building up a larger set of Wick-expanded terms from many individual contributions.

3.21 bad_term_exception

bad_term_exception is an exception type used to report invalid or inconsistent WickTerm objects. It stores the offending term and can therefore be used to provide detailed diagnostics when a contraction cannot be processed.

4 Example workflow

The file `tests/bcs.cpp` demonstrates a complete workflow for using the library in a realistic setting. It builds a simple BCS-like Hamiltonian, applies Wick's theorem to the resulting operator strings, and then evaluates the resulting expressions numerically. The example is intentionally compact, but it exercises most of the library's important building blocks.

The workflow can be summarized as follows.

1. Define symbolic operator strings with Term. Term is used to encode the symbolic structure of the Hamiltonian. The first term represents a bilinear hopping contribution, and the second one encodes the pairing interaction with a coefficient `g(q,p)`. The sums over momenta and spin are stored in SumContainer, while the actual operator sequence is stored in the term's operator vector.

```
const Term H_Kin(1,
    Coefficient("\\epsilon", Momentum('q')),
    SumContainer{ MomentumSum({ 'q' }), Index::Sigma },
    std::vector<Operator>({
        Operator('q', 1, false, Index::Sigma, true),
        Operator('q', 1, false, Index::Sigma, false)
    }));

const Term H_Ph(-1,
    Coefficient::RealInversionSymmetric("g",
        MomentumList({ 'q', 'p' }),
        std::function<void(Coefficient&)>([](Coefficient& coeff) {
            coeff.momenta.sort();
        })),
    SumContainer{ MomentumSum({ 'p', 'q' }) },
    std::vector<Operator>({
        c_k_dagger.with_momentum('q'),
        c_minus_k_dagger.with_momentum('q'),
        c_minus_k.with_momentum('p'),
        c_k.with_momentum('p')
    }));

```

2. Define the contraction patterns that Wick's theorem should recognize. The example uses two templates: one for number operators of the form $c^\dagger_{k\sigma} c_{k\sigma}$

```
    WickOperatorTemplate{
        {IndexComparison{true}},
        Momentum(),
        OperatorType::Number
    }

```

and one for pair operators of the form $c^\dagger_{q\uparrow} c^\dagger_{-q\downarrow}$ and $c_{-q\downarrow} c_{q\uparrow}$.

```
    WickOperatorTemplate{
        {IndexComparison{false, Index::SpinUp, Index::SpinDown}},
        Momentum(),
        OperatorType::SC
```

```
    }

```

These templates are passed to `wicks_theorem`.

3. Evaluate the desired expression (e.g., a commutator), and clean up the result

```
std::vector<Term> commutator_result = commutator(H, right);
clean_up(commutator_result);

std::vector<Term> eight_T = std::vector<Term>{H_Ph * H_Ph};
normal_order(eight_T);
clean_up(eight_T);
```

Note that `commutator` returns a normal ordered result, but using the operator overload `operator*` does not. Hence, `normal_order` must be called in the second example.

4. Transform the resulting expression into `WickTerm` objects via

```
WickTermCollector commutator_wicks;
wicks_theorem(commutator_result, templates, commutator_wicks);
clean_wicks(wicks);
```

The result is typically messy, and should be cleaned up by calling `clean_wicks`. However, the function can do more, if symmetries are known. Those can be applied via the child classes of `WickSymmetry`. Our example adds spin symmetry (expectation values are independent of spin), inversion symmetry (expectation values at $-k$ are the same as at k), and a phase symmetry for the superconducting pair channel (the expectation values of the pair-creation and annihilation operators are real). This reduces the set of equivalent terms and makes the symbolic output more compact.

```
std::vector<std::unique_ptr<WickSymmetry>> symmetries;
symmetries.push_back(std::make_unique<SpinSymmetry>());
symmetries.push_back(std::make_unique<InversionSymmetry>());
symmetries.push_back(std::make_unique<PhaseSymmetry<OperatorType::SC>>());

```

After the definitions above, we can instead use

```
WickTermCollector wicks;
wicks_theorem(H, templates, wicks);
clean_wicks(wicks, symmetries);

```

5. Evaluate the resulting expressions numerically. After the Wick expansion is computed, the example converts the symbolic result into numerical values by assigning momentum values to the summation variables and evaluating the coefficients and contraction operators. Doing so can be done in multiple ways. One example is shown below. Note that this example is kept rather general. If the specifics of the problem are known, one can often heavily abuse them to reduce computational costs.

```
double evaluate_momentum(const Momentum& expression, const std::map<...>&
    momentum_map) {
    double value{};
    for (const MomentumSymbol& momentum : expression) {
        value += momentum.factor * momentum_map.at(momentum.name);
    }
    if (expression.add_Q) {
        value += std::numbers::pi;
    }
    return value;
}
```

```
11
12 double evaluate_coefficients(...){ ... }
13 double evaluate_wick_operators(...){ ... }
14
```